

GAUTAM BUDDHA UNIVERSITY



SCHOOL OF ICT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DETAILED SYLLABI (SEMESTER III-IV)

B.TECH. (COMPUTER SCIENCE AND ENGINEERING)

SPECIALIZATION IN

ARTIFICIAL INTELLIGENCE

ACADEMIC BATCH: 2025-2029

SEMESTER III

A handwritten signature in blue ink, appearing to read 'B. S. S.', is located in the lower-left quadrant of the page.

Scientific Computing with Python			
Course Code:	CAI201	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Develop proficiency in Python for scientific and engineering applications.			
2. Introduce numerical methods and computational techniques for solving scientific problems.			
3. Apply scientific Python libraries for numerical computation, optimization, visualization, and data analysis.			
4. Develop efficient, reproducible, and scalable scientific computing solutions.			
5. Enable students to solve real-world engineering and data-driven computational problems.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Develop Python programs for scientific and engineering computations.			
2. Apply numerical methods using NumPy and SciPy to solve computational problems.			
3. Analyze, visualize, and interpret scientific datasets using Python libraries.			
4. Develop optimization and machine learning solutions for scientific applications.			
5. Design efficient, reproducible, and scalable scientific computing workflows using modern Python tools.			

UNIT I: FOUNDATIONS OF SCIENTIFIC COMPUTING AND PYTHON

Introduction to Scientific Computing, Python programming for scientific applications, Python data structures and control flow, Functions, modules, packages, exceptions, Object-Oriented Programming in Python, Jupyter Notebook environment, Scientific Python ecosystem, Introduction to version control using Git, Libraries: Python, Jupyter, IPython.

UNIT II: NUMERICAL COMPUTING WITH NUMPY AND SCIPY

NumPy arrays and vectorized programming, Matrix operations, Broadcasting, Random number generation, Linear algebra using NumPy, Sparse matrices, Numerical differentiation, Numerical integration, Root finding, Interpolation, Solving ordinary differential equations (ODEs), Libraries: NumPy, SciPy.

UNIT III: SCIENTIFIC DATA ANALYSIS AND VISUALIZATION

Scientific data formats, Data manipulation using Pandas, Data cleaning and pre-processing, Statistical analysis, Exploratory Data Analysis, Scientific visualization, 2D and 3D plotting, Heatmaps, Interactive visualization, Visualization best practices, Libraries: Pandas, Matplotlib, Plotly, Seaborn.

UNIT IV: OPTIMIZATION AND SCIENTIFIC MACHINE LEARNING

Mathematical optimization concepts, Linear and nonlinear optimization, Constrained optimization, Curve fitting, Least-squares estimation, Introduction to scientific machine learning, Feature engineering, Libraries: SciPy Optimize, Scikit learn.

Handwritten signature

36th BoS Date: 30-July-2026

Academic Batch: 2025 and Onwards

UNIT V: ADVANCED SCIENTIFIC COMPUTING

Graph and network analysis, Parallel computing concepts, Performance optimization, Profiling Python code, Introduction to GPU computing, Unit testing, Package development, Reproducible computational research, Applications of scientific computing involving simulation, optimization, or data analysis, Libraries: NetworkX, multiprocessing, Overview of Numba/CuPy, pytest.

Text Books:

1. Hans Petter Langtangen, *A Primer on Scientific Programming with Python* (5th Edition), Springer International Publishing, 2016.
2. Jake VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data* (1st Edition), O'Reilly Media, 2016.
3. Wes McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter* (3rd Edition), O'Reilly Media, 2022.
4. Christian Hill, *Learning Scientific Programming with Python* (2nd Edition), Cambridge University Press, 2020.
5. Eric Matthes, *Python Crash Course: A Hands-On, Project-Based Introduction to Programming* (3rd Edition), No Starch Press, 2023.

Four handwritten signatures in blue ink are visible below the list of text books. The signatures are stylized and appear to be of the same person or a group of people.

Data Structures			
Course Code:	CAI203	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understand the fundamental concepts of data structures, abstract data types, recursion, and algorithm analysis using asymptotic notations.			
2. Analyze, implement, and compare searching, sorting, and hashing techniques based on their efficiency and suitability for different applications.			
3. Design and implement linear data structures such as linked lists, stacks, and queues, and apply them to solve computational problems.			
4. Apply tree-based data structures, including binary trees and binary search trees, to perform efficient storage, traversal, searching, and expression evaluation.			
5. Represent, traverse, and analyze graph data structures using standard graph traversal algorithms for solving connectivity and ordering problems.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Explain the fundamentals of data structures and algorithm analysis.			
2. Implement and analyze searching, sorting, and hashing techniques.			
3. Apply linked lists, stacks, and queues to solve computational problems.			
4. Construct and manipulate tree data structures for efficient data processing.			
5. Represent and traverse graphs using standard graph algorithms.			

UNIT I: INTRODUCTION TO DATA STRUCTURES

Introduction to Data Structures, Abstract Data Types, Linear and Non-linear Data Structures, Arrays as ADT, One Dimensional Array, Basic Operations on Arrays, Recursion: Introduction and Process, Examples: Factorial, Fibonacci Sequence, Tower of Hanoi; Asymptotic analysis introduction, Asymptotic notations, Big-Oh Notation, Time Complexity.

UNIT II: SEARCHING AND SORTING

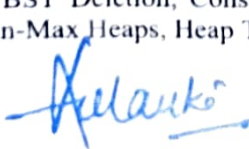
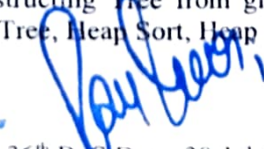
Searching: Linear Search, Binary Search; Sorting: Introduction, Efficiency Considerations, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Counting Sort, Radix Sort, Bucket Sort.

UNIT III: LINKED LIST, STACKS AND QUEUES

Linked List: Introduction, Singly Linked List, Basic Operations on Singly Linked List: Insertion, Deletion, Traversal, Searching, Circular Linked List, Basic Operations on Circular Linked List: Insertion, Deletion, Traversal, Introduction to Doubly Linked List. Stacks: Introduction, Insertion, Deletion, Traversal, Applications: Infix to Postfix Conversion, Post-fix Expression Evaluation, Queues: Introduction, Insertion, Deletion, Traversal, Circular Queue: Insertion, Deletion, Traversal.

UNIT IV: TREES

Tree Data Structures, Height, Complete, Full, Perfect Trees, Binary Trees, Properties of Binary Trees, Representation, Traversal: Pre-Order, In-Order, Post-Order, Binary Search Trees, BST Searching, BST Insertion, BST Deletion, Constructing Tree from given Traversals, Evaluating an expression tree, Heaps, Min-Max Heaps, Heap Tree, Heap Sort, Heap as Priority Queue, Insertion, Deletion.



UNIT V: HASHING AND GRAPHS

Hashing: Hash function, Hash Tables, Hashing Collision Resolution Strategies: Separate Chaining, Linear probing, Quadratic probing, Double hashing; Graphs, Introduction, Different Types of Graphs, Graphs Representations, Incidence Matrix, Adjacency Matrix, Graphs Traversals: Breadth First Search, Depth First Search, Applications: Connected Components, Topological Sort.

Text Books:

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, Introduction to algorithms. MIT Press; 1990 / McGraw-Hill.
2. E. Horowitz, S. Sahni, Fundamentals of data structures, Computer Science Press, 1982.
3. N. Karumanchi, Data Structures and Algorithms Made Easy, CareerMonk Publications, 2017.
4. S. Lipschutz, Data Structures With C, Tata McGraw-Hill, 2011.
5. Aaron M. Tenenbaum, Yeedidyah Langsam, Moshe J. Augenstein, Data Structures Using C, Pearson Education; 2004 / PH.

Handwritten signatures and initials in blue ink.

Machine Learning			
Course Code:	CAI205	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To understand the fundamental concepts, paradigms, and applications of Machine Learning.			
2. To apply supervised and unsupervised learning techniques to solve real-world problems.			
3. To understand the principles of artificial neural networks and deep learning models.			
4. To explore reinforcement learning and recent advancements in machine learning techniques.			
5. To analyze and select appropriate machine learning algorithms for data-driven decision-making.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Explain the concepts, types, and applications of machine learning.			
2. Implement and evaluate supervised learning algorithms for regression and classification tasks.			
3. Develop artificial neural network models using appropriate activation functions and learning algorithms.			
4. Apply unsupervised learning techniques for clustering and dimensionality reduction.			
5. Analyze reinforcement learning, deep learning models, and emerging machine learning techniques for solving practical problems.			

UNIT I: INTRODUCTION TO MACHINE LEARNING

What is Machine Learning, Types of Machine Learning, Supervised Learning, Unsupervised Learning, Reinforcement Learning, Applications of Machine Learning – Stock Price Prediction, Face Recognition, Handwriting Recognition, Image Recognition, Virtual Personal Assistants, Medical Diagnosis, Online Fraud Detection.

UNIT II: SUPERVISED LEARNING (REGRESSION/CLASSIFICATION)

Linear models: Linear Regression, Logistic Regression, Generalized Linear Models; Basic methods: Distance-based methods, Nearest-Neighbours, Decision Trees, Naive Bayes, Support Vector Machines, Nonlinearity and Kernel Methods, Ensemble Methods (Boosting, Bagging, Random Forests).

UNIT III: ARTIFICIAL NEURAL NETWORKS

Biological Neurons and Biological Neural Networks, Artificial Neural Network, Types of Neural Network, Perceptron, History behind Perceptron, Importance of Perceptron, Working of Perceptron, Perceptron Learning, Perceptron Learning Rule, Perceptron Learning of AND & OR gate, XOR gate, Activation functions, Binary Activation function, ReLU, Sigmoid, Hyperbolic, Softmax Activation function, Multilayer Perceptrons, Back propagation Neural Networks, and Feed-Forward Neural Networks, Applications and Future of Neural Networks.

UNIT IV: UNSUPERVISED LEARNING

Clustering: K-means/Kernel K-means, Self-Organizing Maps. Dimensionality Reduction: PCA and kernel PCA, Matrix Factorization and Matrix Completion Generative Models (mixture models and latent factor models).



36th B.O.S. Date: 30-July-2026

Academic Batch: 2025 and Onwards

UNIT V: REINFORCEMENT LEARNING AND DEEP LEARNING

Introduction to Reinforcement Learning, Learning Task, Example of Reinforcement Learning in Practice Deep Learning: Sparse Modeling and Estimation, Modeling Sequence/Time-Series Data, Deep Learning and Feature Representation Learning. Recent trends in various learning techniques of machine learning and classification methods, Case studies in interdisciplinary domains.

Text/ Reference Books:

1. Tom M. Mitchell, Machine Learning, McGraw-Hill Education (India) Private Limited, 2013.
2. Ethem Alpaydin, Introduction to Machine Learning (Adaptive Computation and Machine Learning), The MIT Press 2004.
3. Stephen Marsland, Machine Learning: An Algorithmic Perspective, CRC Press, 2009.
4. Bishop, C., Pattern Recognition and Machine Learning, Berlin: Springer-Verlag.
5. Hastie, T., Tibshirani, R., Friedman, J., The Elements of Statistical Learning, Springer, 2008.
6. James, G., Witten, D., Hastie, T., Tibshirani, R., An Introduction to Statistical Learning, Springer, 2013.

Arulko

Arulko

P

Computer Networks			
Course Code:	CAI207	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Describe how computer networks are organized with the concept of layered approach.			
2. Implement a simple LAN with hubs, bridges and switches.			
3. Analyze the contents in a given Data Link layer packet, based on the layer concept.			
4. Describe what classless addressing scheme is.			
5. Describe how routing protocols work.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Analyze the requirements for a given organizational structure and select the most appropriate networking architecture and technologies.			
2. Have a basic knowledge of the use of cryptography and network security.			
3. Specify and identify deficiencies in existing protocols, and then go onto formulate new and better protocols.			
4. Analyze specify and design the topological and routing strategies for an IP based networking infrastructure.			
5. Have a working knowledge of datagram and internet socket programming.			

UNIT I: INTRODUCTION AND PHYSICAL LAYER

Key concepts of computer network, transmission media, network devices, network topology, topology design issues, types of Network: LAN, MAN, WAN, PAN, ISDN systems and ATM network, OSI-reference model, open system standards, characteristics of network, TCP/IP model, protocols and standards, encoding technique.

UNIT II: SWITCHING AND DATA LINK LAYER

Multiplexing Techniques: TDMA, FDMA, WDMA, CDMA, Data link layer: LLC and MAC sublayers, protocols and design issues, issues IEEE 802 LAN Standards, framing, CRC, error control, flow control, HDLC, ALOHA and performance issues. Frames relay networks and performance parameters.

UNIT III: NETWORK LAYER

Network layer design issues, overview of IPv4 and IPv6, addressing: class full and classless, static and dynamic, subnet and super net, auto configuration through DHCP, routing protocols: RIP, DVR, LSR, OSFP, BGP, congestion control algorithm, subnet concept, virtual LAN, ICMP, multicasting, mobile IP.

UNIT IV: TRANSPORT LAYER

Port addressing schemes, connectionless and connection oriented services: TCP and UDP, wireless TCP, Congestion control, queue management, NAT, PAT, socket format at transport level, socket interface and programming.

UNIT V: APPLICATION LAYER

Client server architecture, domain name services, application services: HTTP, TELNET, RLOGIN, FTP, CBR, NFS, SMTP, POP, IMAP, MIME, voice and video over IP, social issues- privacy, freedom of speech, copy right.

Text Books:

1. S. Tanenbaum, Computer Networks, 4th edition, Prentice Hall, 2008.
2. Forouzan, B.A., Data Communication and Networking, Tata McGraw-Hill.
3. W. Stallings, Data and Computer Communications, 8th edition, Prentice Hall, 2007
4. Douglas E. Comer TCP/IP Principles, Protocols and Architecture, Pearson Education.

W
Stallings Forouzan; P

Operating System			
Course Code:	CAI209	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understanding how Operating System is Important for Computer System.			
2. To make aware of different types of Operating System and their services.			
3. To learn different process scheduling algorithms and synchronization techniques to achieve better performance of a computer system.			
4. To know virtual memory concepts and secondary memory management.			
5. Understanding of Security & protection in Operating System.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Understands the different services provided by Operating System at different level			
2. They learn real life applications of Operating System in every field.			
3. Understands the use of different process scheduling algorithm and synchronization techniques to avoid deadlock.			
4. They will learn different memory management techniques like paging, segmentation and demand paging etc.			
5. Perform implementation of protection mechanisms in operating system			

UNIT I: INTRODUCTION TO OPERATING SYSTEM

Importance of operating systems, basic concepts and terminology about operating system, memory management, processor management, device management, information management functions.

UNIT II: PROCESS MANAGEMENT

Elementary concept of process, job scheduler, process scheduling, operation on process, threads, overview, scheduling criteria, scheduling algorithms, algorithm, and deadlocks: system model, deadlock characterization, deadlocks prevention, deadlocks avoidance, deadlocks detection, recovery from deadlock.

UNIT III: MEMORY MANAGEMENT

Memory management, logical versus physical address space, swapping, contiguous allocation, paging, segmentation, demand paging, page replacement, page replacement algorithms, allocation of frames, thrashing, demand segmentation.

UNIT IV: STORAGE MANAGEMENT

File concept, directory structure, protection, file-system structure, allocation method, free-space management, directory implementation.

UNIT V: I/O SYSTEMS

I/O hardware, Application of I/O interface, Overview of Kernel I/O subsystem, three types of I/O systems, memory based I/O, I/O based I/O, peripheral based I/O.

Text Books:

1. Galvin, Wiley, Operating Systems Concepts, 8th edition, 2009.
2. James L Peterson, Operating Systems Concept, John Wiley & Sons Inc, the 6th edition, 2007.

36th BOS Date: 19 July 2026

Academic Batch: 2025 and Onwards

Reference Books:

1. Deitel H. M., An Introduction to Operating Systems, Addison-Wesley, 1990.
2. Stallings William, Operating Systems, PHI, New Delhi, 1997.
3. S. Tanenbaum Modern Operating Systems, Pearson Education, 3rd edition, 2007.
4. Nutt, Operating System, Pearson Education, 2009.
5. S. Tanenbaum, Distributed Operating Systems, Prentice Hall, 2nd edition, 2007.

Handwritten signatures in blue ink:
A large, stylized signature on the left, followed by a smaller signature in the middle, and a small circular mark on the right.

Scientific Computing with Python Lab			
Course Code:	CAI281	Course Credits:	2
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Develop proficiency in using Python and scientific computing libraries for solving computational problems. 2. Apply numerical methods and scientific programming techniques to analyze and solve engineering and scientific problems. 3. Perform data processing, visualization, and optimization using modern Python tools and libraries. 4. Design, implement, and evaluate scientific computing solutions through practical experimentation and real-world applications. 5. Develop analytical, programming, and problem-solving skills for scientific and engineering computations.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Develop Python programs using scientific computing libraries such as NumPy, SciPy, Pandas, and Matplotlib for computational tasks. 2. Apply numerical methods and matrix computations to solve scientific and engineering problems. 3. Perform data analysis, visualization, and optimization using appropriate Python libraries. 4. Design, implement, and evaluate scientific computing solutions for real-world engineering and data-driven applications. 5. Analyze computational performance and develop efficient, reliable, and reproducible scientific computing programs.			

LIST OF EXPERIMENTS:

1. Write a Python program to perform basic scientific computations using variables, operators, functions, and control statements in Jupyter Notebook.
2. Write a Python program to create and manipulate NumPy arrays, perform vectorized operations, and compare their performance with Python lists.
3. Implement matrix operations (addition, multiplication, transpose, inverse, and determinant) using the NumPy library.
4. Write a Python program to solve systems of linear equations and compute eigenvalues and eigenvectors using NumPy library.
5. Implement numerical methods for root finding and numerical integration using the SciPy library.
6. Write a Python program to perform interpolation and solve ordinary differential equations (ODEs) using SciPy library.
7. Write a Python program to load, clean, manipulate, and analyze datasets using the Pandas library.
8. Develop Python programs to create line plots, bar charts, scatter plots, histograms, and heatmaps using Matplotlib and Seaborn library.
9. Implement curve fitting, least-squares estimation, and optimization problems using the SciPy Optimize module.
10. Write a Python program to develop and evaluate regression and classification models using the Scikit-learn library.

Handwritten signature

Date: 30 July 2026

Academic Batch: 2025 and Onwards

11. Implement a Python program to profile computational performance, compare execution time of algorithms, and optimize code using vectorization techniques.

Handwritten signatures in blue ink:
A large signature on the left, a large signature on the right, and a small circular stamp or mark below the right signature.

Data Structures Lab			
Course Code:	CAI283	Course Credits:	2
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understand the implementation of fundamental data structures.			
2. Develop programs using linear and non-linear data structures.			
3. Apply appropriate data structures to solve computational problems.			
4. Analyze the performance of data structures and algorithms.			
5. Enhance programming, debugging, and problem-solving skills.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Implement array operations, recursion, and basic algorithm analysis.			
2. Develop and evaluate searching, sorting, and hashing algorithms.			
3. Implement linked lists, stacks, and queues for practical applications.			
4. Construct and manipulate tree and graph data structures using standard algorithms.			
5. Design efficient solutions by selecting appropriate data structures for real-world problems.			

List of Experiments:

1. Implement one-dimensional and two-dimensional array operations (insertion, deletion, searching, traversal, merging).
2. Implement recursive algorithms for Factorial and Fibonacci series. Analyze their time complexity.
3. Implement Linear Search and Binary Search. Compare their execution time and efficiency for different input sizes.
4. Implement Bubble Sort, Selection Sort, and Insertion Sort. Compare their time complexities experimentally.
5. Implement Merge Sort and Quick Sort. Compare recursive divide-and-conquer sorting algorithms.
6. Implement Heap Sort and Counting Sort algorithms.
7. Implement Hash Tables using Separate Chaining and Open Addressing.
8. Implement Singly and Circular Linked Lists with insertion, deletion, searching, and traversal operations.
9. Implement Stack using Arrays and Linked Lists.
10. Implement Circular Queue and Priority Queue using Arrays and Linked Lists.
11. Implement Binary Tree and Binary Search Tree operations including insertion, deletion, searching, and traversals (Preorder, Inorder, Postorder).
12. Implement Graph representations using Adjacency Matrix and Adjacency List. Perform BFS, DFS.

Computer Networks Lab			
Course Code:	CAI285	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Lectures (L + T):	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Practical knowledge of working principles of various communication protocols.			
2. Analyze structure and formats of TCP/IP layer protocols.			
3. Understanding of networking fundamentals.			
4. Understanding of learning the process of Internet of Things applications planning.			
5. Understanding of configuration of various end devices, server, routers and switches.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Understand the practical approach to network communication protocols.			
2. Understand network layers, structure/format and role of each network layer.			
3. Able to design and implement various network application such as data transmission between client and server, file transfer, real-time multimedia transmission.			
4. Understand the various Routing Protocols/Algorithms and Internetworking.			
5. Learn to configure server.			

List of Experiments:

1. Introduction to transmission media (CAT5, OFC, COAXIAL CABLE Wireless).
2. Introduction to network interfaces (Wired and Wireless).
3. Configure and installing an Ethernet (10/100).
4. Performance evaluation of Ethernet (10/100).
5. Topology design (Ring, Bus).
6. Generation of data packet and measurement (CBR, VBR, Poison).
7. Implement the following:
 - a. Router Configuration
 - b. Server Configuration
 - c. Server Configuration
8. Congestion control of network and QoS of network
9. Protocols and the configuration
10. Security (WEP, WPA) and Qualnet

Machine Learning Lab			
Course Code:	CAI287	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Lectures + Tutorials (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Lectures (L + T):	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Develop proficiency in Python programming and machine learning libraries for data analysis.			
2. Implement and evaluate supervised and unsupervised machine learning algorithms.			
3. Apply appropriate data pre-processing, visualization, and model evaluation techniques.			
4. Perform model optimization through hyperparameter tuning and comparative analysis.			
5. Solve real-world machine learning problems using standard tools and datasets.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Use Python libraries to perform data pre-processing, statistical analysis, and visualization.			
2. Implement and evaluate regression and classification models using appropriate performance metrics.			
3. Apply hyperparameter tuning techniques to improve the performance of machine learning models.			
4. Implement clustering techniques for document and data analysis using suitable evaluation measures.			
5. Develop complete machine learning solutions for real-world datasets using standard Python-based frameworks.			

List of Experiments:

1. Implement a Python program to compute measures of central tendency (Mean, Median, and Mode) and measures of dispersion (Variance and Standard Deviation).
2. Implement basic functions of Python libraries such as statistics, math, NumPy, and SciPy for scientific computing.
3. Implement basic functions of Python libraries for machine learning applications, including Pandas and Matplotlib, for data manipulation and visualization.
4. Implement Linear Regression using five datasets of varying sizes. Compare the model performance by plotting evaluation metrics such as MAE, MSE, RMSE, and R^2 Score.
5. Implement the K-Nearest Neighbors (KNN) classifier using scikit-learn. Evaluate the model on five datasets of varying sizes by comparing Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Using one large dataset, vary the value of K (1, 3, 5, 7, 9, 11, 21, and 31) and compare the performance using the same evaluation metrics.
6. Implement the Decision Tree classifier using scikit-learn and perform hyperparameter tuning. Evaluate the model on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
7. Implement the Naïve Bayes classifier using scikit-learn. Evaluate the model on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

8. Implement the Support Vector Machine (SVM) classifier using scikit-learn. Evaluate the model on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
9. Implement the Random Forest classifier using scikit-learn. Evaluate the model on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
10. Implement ensemble learning algorithms (AdaBoost and XGBoost). Evaluate the models on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
11. Implement an Artificial Neural Network (ANN) using scikit-learn or TensorFlow/Keras and perform hyperparameter tuning. Evaluate the model on five datasets of varying sizes and compare Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
12. Implement K-Means clustering for document clustering using suitable text datasets. Evaluate the clustering performance using Silhouette Score and visualize the resulting clusters.



SEMESTER IV

A handwritten signature in blue ink, appearing to read 'V. S. S.', is written over a horizontal line.

Design and Analysis of Algorithms			
Course Code:	CAI202	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To analyze the asymptotic time and space performance of algorithms using appropriate complexity measures.			
2. To develop rigorous correctness proofs for algorithms using formal techniques such as invariants and induction.			
3. To acquire and demonstrate familiarity with fundamental and advanced algorithms and data structures.			
4. To apply key algorithmic design paradigms and analytical methods to solve computational problems effectively.			
5. To synthesize efficient and scalable algorithmic solutions for common engineering and real-world problem scenarios.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Analyze algorithms and data structures using asymptotic notations, recurrence relations, and amortized analysis to evaluate time and space complexity.			
2. Design efficient algorithms using appropriate paradigms such as greedy, divide-and-conquer, dynamic programming, backtracking, and branch and bound.			
3. Implement and apply advanced data structures and graph algorithms to solve complex computational problems.			
4. Design and evaluate advanced algorithmic solutions using randomized, approximation, and online paradigms with respect to correctness, efficiency, and practical applicability.			
5. Classify computational problems into complexity classes (P, NP, NP-Complete, NP-Hard) and justify algorithmic choices for intractable problems.			

UNIT I: INTRODUCTION

Introduction to Algorithm, Time Complexity and Space Complexity, Order of Growth, Asymptotic Notations: Big Oh, Theta, Omega, Amortized analysis, Analysing control statement, Loop Invariant, Recurrence Relations: Introduction, Substitution Method, Recursion Tree Method, Master's Theorem. Searching and Sorting Algorithms analysis.

UNIT II: ALGORITHM DESIGN TECHNIQUES

Divide and Conquer: Binary Search, Merge Sort, Strassen's Matrix Multiplication; Greedy Algorithm: General Characteristics, Knapsack Problem, Huffman code; Dynamic Programming: Introduction, Principle of Optimality, 0-1 Knapsack, Longest Common Subsequence, Matrix Chain Multiplication.

UNIT III: STATE SPACE SEARCH ALGORITHMS AND PATTERN MATCHING

Backtracking: State-Space Search Tree, N-Queen's problem, Graph Colouring; Branch and Bound: Travelling Salesman Problem; Pattern Matching: The Naïve string-matching algorithm, Rabin-Karp algorithm, Knuth-Morris-Pratt algorithm.

Handwritten signatures and initials in blue ink.

UNIT IV: ADVANCED GRAPH ALGORITHMS

Minimum Spanning Trees: Prim's algorithm, Kruskal's algorithm; Shortest Path: Bellman-Ford algorithm, Dijkstra's algorithm, Floyd Warshall algorithm; Network flow: Flow networks, Max-Flow, Min-Cut, Ford-Fulkerson method.

UNIT V: ADVANCED ALGORITHMS

P and NP, NP Complete and NP-Hard; Advanced Data Structures and their applications: Height Balanced Trees: AVL Trees, Balance Factor, Rotations, Insertion, Deletion, B-Trees, Insertion, Deletion, Advanced Algorithms and their applications.

Text Books:

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 4th Edition, MIT Press.
2. Anany Levitin, "Introduction to the Design and Analysis of Algorithm", Pearson Education Asia, 2003.
3. Sara Baase and Allen Van Gelder, "Computer Algorithms-Introduction to the Design and Analysis", Pearson Education Asia, 2003.
4. A. V. Aho, J.E. Hopcroft and J.D. Ullman, "The Design and Analysis of Computer Algorithms", Pearson Education Asia, 2003.

Database Management System			
Course Code:	CAI204	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Describe the fundamental elements of relational database management systems			
2. Explain the basic concepts of relational data model, entity-relationship model, relational database design, relational algebra and SQL.			
3. Design ER-models to represent simple database application scenarios			
4. Convert the ER-model to relational tables, populate relational database and formulate SQL queries on data.			
5. Improve the database design by normalization.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Students will have an understanding of database concepts and thorough knowledge of database software's.			
2. Students will be able to model an application's data requirements using ER diagrams			
3. Students will be able to write SQL commands to create tables and query data in a relational DBMS			
4. Students will be able to execute various advanced SQL queries related to transactions, concurrency			
5. Students will be able to explain the principle of transaction management design.			

UNIT I: DATABASE SYSTEM

Data base system vs. file system, view of data, data abstraction, instances and schemas, data models, ER model, relational model, DDL, DML, database access for applications programs, data base users and administrator, transaction management, data base system structure, storage manager, query processor, history of data base systems, data base design and ER diagrams, beyond ER design entities, attributes and entity sets, relationships and relationship sets, additional features of ER model, concept design with the ER model, and conceptual design for large enterprises.

UNIT II: RELATIONAL MODEL

Introduction to the relational model, integrity constraint over relations, enforcing integrity constraints, querying relational data, and logical data base design, destroying /altering tables and views. relational algebra and calculus: relational algebra, selection and projection set operations, renaming, joins, division.

UNIT III: BASIC SQL QUERY

Examples of basic SQL queries, nested queries, correlated nested queries set, comparison operators, aggregative operators, NULL values, comparison using null values, logical connectivity's, AND, OR and NOT, impact on SQL constructs, outer joins, disallowing NULL values, complex integrity constraints in SQL triggers and active data bases.

Handwritten signatures and initials in blue ink.

UNIT IV: SCHEMA REFINEMENT

Problems caused by redundancy, decompositions, problem related to decomposition, reasoning about FDS, FIRST, SECOND, THIRD normal form, BCNF, forth normal form, lossless join decomposition, dependency preserving decomposition, schema refinement in data base design, multi valued dependencies.

UNIT V: OVERVIEW OF TRANSACTION MANAGEMENT

ACID properties, transactions and schedules, concurrent execution of transaction, lock based concurrency control, performance locking, and transaction support in SQL, crash recovery, concurrency control, Serializability and recoverability, lock management, lock conversions, dealing with deadlocks, specialized locking techniques, concurrency without locking, crash recovery: ARIES, log, other recovery related structures, the write, ahead log protocol, check pointing, recovering from a system crash, media recovery, other approaches and interaction with concurrency control.

Text Books:

1. Elmasri Navrate, Data Base Management System, Pearson Education, 2008.
2. Raghurama Krishnan, Johannes Gehrke, Data Base Management Systems, TMH, 3rd edition, 2008.
3. C. J. Date, Introduction to Database Systems, Pearson Education, 2009.
4. Silberschatz, Korth, Database System Concepts, McGraw hill, 5th edition, 2005.
5. Rob, Coronel & Thomson, Database Systems Design: Implementation and Management.

Deep Learning			
Course Code:	CAI206	Course Credits:	3
Course Category:	PCC	Course (U / P):	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understand the basics of artificial neural networks and deep neural networks.			
2. Implement deep neural networks with optimization.			
3. Build basic deep learning models and analyze the results.			
4. Design and implement convolution neural networks.			
5. Design and implement sequential and advanced deep architectures.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Understand and implement artificial neural networks and deep neural networks.			
2. Analyze the given dataset for designing a neural network-based solution.			
3. Build basic deep learning models and interpret the results.			
4. Design and implement convolution neural networks.			
5. Design and implement sequential and advanced deep architectures to solve real world problems.			

UNIT I: INTRODUCTION

Basics of artificial neural networks (ANN): Artificial neurons, Computational models of neurons, Architecture of neural networks, Feedforward neural networks, Backpropagation learning, Cost Function, Regularization, Autoencoders. Logistic Regression vs neural networks, Activation functions, L-layer NN. Optimization: Types of errors, bias-variance trade-off, overfitting-underfitting, brief review of concepts from optimization, variants of gradient descent, momentum-based methods.

UNIT II: DEEP NEURAL NETWORKS

Deep neural networks (DNNs): Difficulty of training DNNs, Bias-Variance Trade-off, Optimization: Optimization algorithms for neural networks (AdaGrad, RMSProp, Adam), Hyperparameters tuning (logarithmic scale), Regularization methods (dropout, drop connect, batch normalization), Multi-class classification.

UNIT III: CONVOLUTION NEURAL NETWORKS

Convolution neural networks (CNNs): Introduction to CNNs – convolution, pooling, Kernel filter, Principles behind CNNs, Deep CNNs, Different deep CNN architectures – LeNet, AlexNet, VGG, ResNet, Training CNNs, Understanding and visualizing CNNs.

UNIT IV: SEQUENTIAL MODELS

Recurrent neural networks (RNNs): Sequence modelling using RNNs, Back propagation through time, Bidirectional RNNs, Gated RNN Architecture, Long Short-Term Memory (LSTM), Bidirectional LSTMs.

UNIT V: ADVANCED DEEP ARCHITECTURES

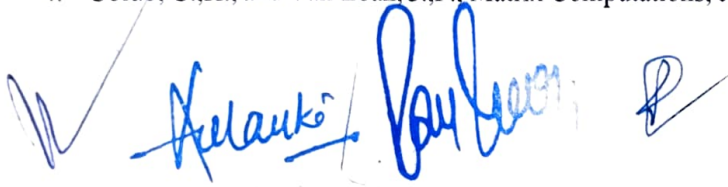
Generative models: Restrictive Boltzmann Machines (RBMs), Stacking RBMs, Belief nets, Generative Adversarial Networks (GANs). Applications: Applications in vision, speech and natural language processing.

Handwritten signature
36th BoS Date: 30 July 2026

Academic Batch: 2025 and Onwards

Text/Reference Books:

1. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press, 2016.
2. Bishop, C. ,M., Pattern Recognition and Machine Learning, Springer, 2006.
3. Yegnanarayana, B., Artificial Neural Networks PHI Learning Pvt. Ltd, 2009.
4. Golub, G.,H., and Van Loan,C.,F., Matrix Computations, JHU Press, 2013.

The image shows four handwritten signatures in blue ink. The first signature is a stylized 'N'. The second signature appears to be 'Helanki'. The third signature is 'Van Loan'. The fourth signature is a stylized 'P'.

Theory of Automata			
Course Code:	CAI208	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To introduce the fundamental concepts of automata, formal languages, and computability.			
2. To develop understanding of finite automata and regular expressions for pattern recognition and lexical analysis.			
3. To study context-free grammars and pushdown automata for syntax analysis and language processing.			
4. To understand the concepts of Turing machines, decidability, and computability.			
5. To provide theoretical foundations required for compiler design, artificial intelligence, and advanced computing systems.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Design deterministic and non-deterministic finite automata for regular languages.			
2. Apply regular expressions and finite automata for pattern matching and lexical analysis.			
3. Construct context-free grammars and pushdown automata for context-free languages.			
4. Analyze formal languages using pumping lemmas and closure properties.			
5. Design Turing machines and explain the concepts of decidability and computability.			
6. Relate automata theory concepts with applications in compiler design, natural language processing, and artificial intelligence.			

UNIT I: INTRODUCTION

Alphabets, Strings and Languages; Automata and Grammars, Deterministic finite Automata (DFA)- Formal Definition, Simplified notation: State transition graph, Transition table, Language of DFA, Nondeterministic finite Automata (NFA), NFA with epsilon transition, Language of NFA, Equivalence of NFA and DFA, Minimization of Finite Automata, Distinguishing one string from other, Myhill-Nerode Theorem.

UNIT II: REGULAR EXPRESSION (RE)

Definition, Operators of regular expression and their precedence, Algebraic laws for Regular expressions, Kleene's Theorem, Regular expression to FA, DFA to Regular expression, Arden Theorem, Non-Regular Languages, Pumping Lemma for regular Languages. Application of Pumping Lemma, Closure properties of Regular Languages, Decision properties of Regular Languages, FA with output: Moore and Mealy machine, Equivalence of Moore and Mealy Machine, Applications and Limitation of FA.

UNIT III: CONTEXT FREE GRAMMAR (CFG) AND CONTEXT FREE LANGUAGES (CFL)

Context free grammar (CFG) and Context Free Languages (CFL): Definition, Examples, Derivation, Derivation trees, Ambiguity in Grammar, Inherent ambiguity, Ambiguous to Unambiguous CFG, Useless symbols, Simplification of CFGs, Normal forms for CFGs: CNF and GNF, Closure properties of CFLs, Decision Properties of CFLs: Emptiness, Finiteness and Membership, Pumping lemma for CFLs.

UNIT IV: PUSH DOWN AUTOMATA (PDA)

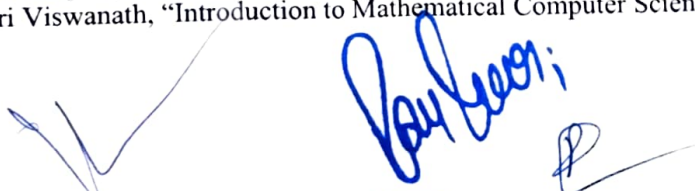
Description and definition, Instantaneous Description, Language of PDA, Acceptance by Final state, Acceptance by empty stack, Deterministic PDA, Equivalence of PDA and CFG, CFG to PDA and PDA to CFG, two stack PDA.

UNIT V: TURING MACHINES (TM)

Basic model, definition and representation, Instantaneous Description, Language acceptance by TM, Variants of Turing Machine, TM as Computer of Integer functions, Universal TM, Church's Thesis, Recursive and recursively enumerable languages, Halting problem.

Text/Reference Books:

1. Hopcroft, Ullman, "Introduction to Automata Theory, Languages and Computation", Pearson Education.
2. Peter Linz, "An Introduction to Formal Language and Automata", Narosa Publishing house.
3. YN Singh "Mathematical Foundation of Computer Science", New Age International.
4. Malviya, AK "Theory of Computation and Application", BPaperback Publications
5. Papadimitrou, C. and Lewis, CL, "Elements of the Theory of Computation", Pearson Publication.
6. K. Krithivasan and R. Rama; Introduction to Formal Languages, Automata Theory and Computation; Pearson Education.
7. KLP Mishra and N. Chandrasekaran, "Theory of Computer Science: Automata, Languages and Computation", PHI Learning Private Limited, Delhi India.
8. Harry R. Lewis and Christos H. Papadimitriou, Elements of the theory of Computation, Second Edition, Prentice-Hall of India Pvt. Ltd.
9. Micheal Sipser, "Introduction of the Theory and Computation", Thomson Learning.
10. Katuri Viswanath, "Introduction to Mathematical Computer Science, An" Universities Press.



Digital Systems and Computer Architecture			
Course Code:	CAI210	Course Credits:	3
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To understand the fundamentals of digital logic, Boolean algebra, number systems, data representation, and arithmetic operations used in digital computers.			
2. To analyze and design combinational and sequential logic circuits using Boolean minimization techniques and standard digital building blocks.			
3. To understand the organization and operation of computer processors, instruction set architectures, and control unit design.			
4. To explain memory hierarchy, cache organization, pipelining, and input/output organization in modern computer systems.			
5. To develop a strong foundation in digital system design and computer organization for advanced studies in computer architecture, embedded systems, operating systems, and related domains.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Explain digital logic fundamentals, Boolean algebra, and data representation techniques.			
2. Design and analyze combinational and sequential digital circuits.			
3. Describe processor organization, instruction execution, and instruction set architectures.			
4. Analyze memory hierarchy, control unit organization, and pipelined processing.			
5. Explain input/output organization and the fundamentals of parallel and multicore architectures.			

UNIT I: Digital Logic Fundamentals and Data Representation

Introduction to Digital Systems, Analog and Digital Systems, Number Systems (Binary, Octal, Decimal, Hexadecimal), Base Conversions, Binary Codes (BCD, Gray Code, ASCII), Signed Number Representation (1's and 2's Complement), Fixed-Point and Floating-Point Representation, Binary Arithmetic, Boolean Algebra, Laws of Boolean Algebra, DeMorgan's Theorems, Logic Gates, Universal Gates, NAND/NOR Realization of Boolean Functions, Boolean Functions, Canonical Forms (SOP and POS), Truth Tables, Register Transfer Language (RTL), Register Transfer and Micro-operations.

UNIT II: Boolean Function Minimization and Combinational Logic Design

Boolean Function Minimization using Karnaugh Maps (3, 4 and 5 Variables), Don't Care Conditions, Quine-McCluskey Minimization Technique, Design of Combinational Circuits, Encoders, Decoders, Multiplexers, Demultiplexers, Multiplexers as Boolean Function Generators, Half and Full Adders, Ripple Carry Adder, Carry Look-Ahead Adder, Binary Subtractors, Magnitude Comparators, Arithmetic Logic Unit (ALU), Basic Arithmetic Algorithms: Binary Multiplication and Division Algorithms.

UNIT III: Sequential Logic and Processor Organization

Latches (SR and D), Flip-Flops (SR, JK, D and T), Characteristic Tables, Registers, Shift Registers, Ripple and Synchronous Counters, Finite State Machines (Mealy and Moore Models), State Diagrams, Functional Units of a Computer, Basic Computer Organization, Bus Structures and Interconnections,

CPU and Register Organization, Instruction Cycle, Instruction Formats, Addressing Modes, Instruction Set Architecture (ISA), Instruction Cycle and Instruction Execution, RISC and CISC Architectures.

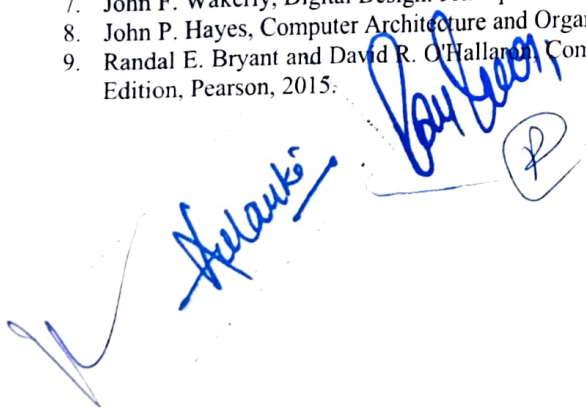
UNIT IV: Control Unit and Memory Organization

Control Unit Design, Hardwired and Microprogrammed Control Units, Memory Hierarchy, Semiconductor Memories (SRAM and DRAM), Memory Chip Organization, Main Memory, Cache Memory, Cache Mapping Techniques and Cache Performance, Associative Memory, Virtual Memory, Instruction Pipelining, Pipeline Hazards.

UNIT V: Input/Output Organization and Computer Architecture

Input/Output Organization, Programmed I/O, Interrupt-Driven I/O, Direct Memory Access (DMA), I/O Interfaces, Serial and Parallel Communication, PCI Express (PCIe) and USB Interfaces, Input and Output Devices, Secondary Storage Devices, Flynn's Classification, Introduction to Parallel Processing and Multicore Architectures.

Text/ Reference Books:

1. M. Morris Mano and Michael D. Ciletti, Digital Design, 6th Edition, Pearson, 2018.
 2. David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 6th Edition, Morgan Kaufmann, 2021.
 3. William Stallings, Computer Organization and Architecture: Designing for Performance, 11th Edition, Pearson, 2022.
 4. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian, Computer Organization and Embedded Systems, 6th Edition, McGraw-Hill, 2012.
 5. Charles H. Roth Jr. and Larry L. Kinney, Fundamentals of Logic Design, 8th Edition, Cengage Learning, 2021.
 6. Andrew S. Tanenbaum and Todd Austin, Structured Computer Organization, 6th Edition, Pearson, 2013.
 7. John F. Wakerly, Digital Design: Principles and Practices, 5th Edition, Pearson, 2018.
 8. John P. Hayes, Computer Architecture and Organization, 3rd Edition, McGraw-Hill.
 9. Randal E. Bryant and David R. O'Hallaron, Computer Systems: A Programmer's Perspective, 3rd Edition, Pearson, 2015.
- 

Design and Analysis of Algorithms Lab			
Course Code:	CAI282	Course Credits:	2
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Lectures (L + T):	10	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Implement and experimentally analyze algorithms using appropriate measures of time and space complexity.
2. Develop solutions using divide-and-conquer, greedy, dynamic programming, and backtracking paradigms.
3. Implement tree, heap, and graph structures required for efficient algorithmic problem solving.
4. Evaluate alternative algorithms on varied input characteristics and justify an appropriate choice.
5. Develop correct, readable, and testable programs using C, Java, or Python.

COURSE OUTCOMES

At the end of the course the students should be able to:

1. Implement algorithms and empirically analyze their best-, average-, and worst-case behaviour.
2. Design and implement divide-and-conquer and greedy algorithms for sorting, selection, spanning tree, shortest path, scheduling, coding, and knapsack problems.
3. Formulate and implement dynamic programming and backtracking solutions and reconstruct the corresponding optimal or feasible solution.
4. Implement and apply heaps, search trees, balanced trees, and graph traversal techniques.
5. Implement standard string-matching algorithms and compare competing algorithms using reproducible experiments.

PRACTICALS

Note: Students may use C, Java, or Python. Each experiment must include algorithm/pseudocode, source code, test cases, output, and a brief complexity/performance analysis.

Lab 1: Experimental Analysis of Algorithms

1. Implement iterative and recursive versions of factorial and Fibonacci computation; count basic operations and compare their running times for increasing input sizes.
2. Implement linear search and binary search; test best-, average-, and worst-case inputs and plot input size versus execution time.

Lab 2: Elementary Sorting and Heap-Based Processing

1. Implement bubble sort, insertion sort and selection sort; record comparisons and data movements for sorted, reverse-sorted, and random inputs.

Lab 3: Divide-and-Conquer Fundamentals

1. Implement binary search using both iterative and recursive approaches; compare their execution time and auxiliary space.
2. Implement merge sort; test it on arrays containing duplicate, negative, sorted, reverse-sorted, and random values, and analyze its time and auxiliary space.

Lab 4: Advanced Divide-and-Conquer Algorithms

1. Implement quick sort using first-element, last-element, randomized, and median-of-three pivot selection; compare performance on different input distributions.
2. Implement Strassen's matrix multiplication and compare its correctness and execution time with conventional matrix multiplication for compatible matrix sizes.

Lab 5: Greedy Algorithms for Spanning Trees and Scheduling

1. Implement Prim's algorithm to obtain a minimum spanning tree of a weighted, connected, undirected graph and report its total cost.
2. Implement Kruskal's algorithm using disjoint-set operations; compare its output and running time with Prim's algorithm on sparse and dense graphs.

Lab 6: Greedy Shortest Paths, Coding, and Knapsack

1. Implement Dijkstra's algorithm for a graph with non-negative edge weights; display the shortest distance and path from a selected source to every vertex.
2. Implement the fractional knapsack problem; display the selected fraction of each item and compare the result with a simple value-based selection strategy.

Lab 7: Dynamic Programming - Paths and Knapsack

1. Implement the 0/1 knapsack problem using memoization and tabulation; reconstruct the selected items and compare the two implementations.
2. Implement the Floyd-Warshall algorithm for all-pairs shortest paths; construct the distance matrix and identify whether a negative cycle exists.

Lab 8: Dynamic Programming and Backtracking

1. Implement the traveling salesperson problem using dynamic programming for small instances; report the minimum tour cost and the tour.
2. Implement the N-Queens problem using backtracking; generate all valid solutions for a given board size and report the number of states explored.

Lab 9: String Matching and Comparative Study

1. Implement the Naive and Rabin-Karp string-matching algorithms; report all match positions, spurious hits, and character comparisons.
2. Implement the Knuth-Morris-Pratt algorithm; construct and display the prefix table and verify matches for patterns with repeated prefixes.

Lab 10: Advanced Trees and Graph Traversal

1. Implement a binary search tree with insertion, deletion, search, and inorder, preorder, and postorder traversals; verify the BST property after updates.
2. Implement an AVL tree with all required rotations; compare its height and search cost with an ordinary BST for the same insertion sequence.

Database Management System Lab			
Course Code:	CAI284	Course Credits:	2
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To explain basic database concepts, applications, data models, schemas and instances.			
2. To demonstrate the use of constraints and relational algebra operations.			
3. To emphasize the importance of normalization in databases.			
4. To facilitate students in Database design			
5. To familiarize issues of concurrency control and transaction management.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Students get practical knowledge on designing and creating relational database systems.			
2. Understand various advanced queries execution such as relational constraints, joins, set operations, aggregate functions, trigger, views and embedded SQL			
3. Design a commercial relational database system (Oracle, MySQL) by writing SQL using the system			
4. Use the basics of SQL and construct queries using SQL in database creation and interaction.			
5. Analyze and Select storage and recovery techniques of database system.			

List of Experiments:

1. Introduction to MySQL, an exercise of data types in MySQL & Data Definition Language Commands.
2. Exercise on Data Manipulation Language and Transaction Control Commands.
3. Exercise of Type of Data Control Commands.
4. Exercise of joins (Single Table - Using Normalization)
5. Exercise of joins (Multiple Table - Using Normalization)
6. Exercise on GROUP BY/ORDER BY Clause and Date Arithmetic.
7. Exercise on different Functions (Aggregate, Math and String).
8. Exercise on different types of sub queries.
9. Exercise on Procedures.
10. Exercise on Views.

Handwritten signature: Anil Kumar

Handwritten signature: Anil Kumar

Handwritten signature: Anil Kumar

Deep Learning Lab			
Course Code:	CAI286	Course Credits:	2
Course Category:	PCC	Course (U / P):	U
Course Year (U / P):	2U	Course Semester (U / P):	4U
No. of Lectures + Tutorials (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Lectures (L + T):	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Implement the various deep learning algorithms in Python.			
2. Learn to work with different deep learning frameworks like Keras, Tensor flow, PyTorch..			
3. Understand complexity of Deep Learning algorithms and be capable of performing distributed computations.			
4. Be capable of confidently applying common Deep Learning algorithms in practice and implementing their own Deep Learning algorithms.			
5. Be capable of performing experiments in Deep Learning using real-world data.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Implement deep learning algorithms and understand neural networks.			
2. Understand concepts of TensorFlow, its main functions, operations and the execution pipeline.			
3. Learn to implement and apply convolution neural networks for solving real-word problems.			
4. Learn to implement and apply recurrent neural networks for solving real-word problems.			
5. Implement and improve deep learning models in TensorFlow and interpret the results.			

List of Practicals

1. To implement Logistic Regression as a neural network for classification problem.
2. To implement one hidden layered neural network for planar data classification.
3. To implement deep neural network with forward and backward propagation.
4. To implement deep neural network for image classification.
5. To implement various advanced optimization methods.
6. To build Sign language classifier using neural network in TensorFlow library.
7. To build Sign language classifier using the TensorFlow CNN Convolutional Neural Networks.
8. To implement Emotion Detection in Images of Faces using Keras.
9. To implement the ResNets for image classification.
10. To implement Character level language model using RNNs.
11. To implement a deep learning model using LSTM to generate music.
12. To implement word vector representations to build an Emojifier.